

API_REQUIREMENTS

Kju Backend — API Requirements

Complete specification of every endpoint the frontend needs. No hardcoded data should remain in the frontend after integration.

Base URL: `http://localhost:8000/api`

Auth: No session tokens. Login returns a `doctorId` — pass it in the request body wherever the backend needs to know which doctor is acting.

Content-Type: `application/json` (unless stated otherwise)

Date format: ISO 8601 (`2026-02-10T09:30:00Z`)

Table of Contents

1. [Login](#)
 2. [Notifications](#)
 3. [Patients](#)
 4. [Chat / Knowledge Graph](#)
 5. [File Uploads](#)
 6. [Diagnostics Pipeline](#)
 7. [Report Operations](#)
-

1. Login

No tokens, no sessions. Just validate credentials and return the doctor object. The frontend stores the `doctorId` in memory/localStorage and passes it in subsequent requests.

1.1 `POST /login`

Validate the medical license ID + password. On success, return the doctor's profile.

Request Body:

```
{
  "medicalLicenseId": "ML-2024-8821",
  "password": "somepassword"
}
```

Success Response (200):

```
{
  "success": true,
  "doctor": {
    "id": "doc-001",
    "name": "Dr. Sarah Chen",
    "specialty": "Internal Medicine",
    "medicalLicenseId": "ML-2024-8821"
  }
}
```

Error Response (401):

```
{
  "error": "Invalid credentials",
  "code": "AUTH_FAILED"
}
```

Frontend location: `LoginPage.tsx` — replaces the `setTimeout` fake login at `handleLogin`. On success, store the returned `doctor` object (including `doctor.id`) in app state / localStorage. This `doctor` object is used by `Header.tsx` and `DashboardPage.tsx` (replaces `mockDoctor`). Logout is purely frontend — just clear the stored doctor and navigate to `/`.

2. Notifications

2.1 `POST /notifications`

Fetch all notifications for a doctor.

Request Body:

```
{
  "doctorId": "doc-001"
}
```

Success Response (200):

```
{
  "notifications": [
    {
      "id": "notif-001",
      "type": "urgent",
      "message": "Critical vitals detected for Robert Kim",
      "patientId": "pat-003",
      "timestamp": "2026-02-10T09:30:00Z"
    },
    {
      "id": "notif-002",
      "type": "warning",
      "message": "Blood pressure elevated for Maria Garcia",
      "patientId": "pat-002",
      "timestamp": "2026-02-10T08:15:00Z"
    }
  ]
}
```

type enum values: `"urgent"` | `"info"` | `"warning"`

Frontend location: `DashboardPage.tsx` → passed to `Header.tsx` — replaces `mockNotifications`.

3. Patients

3.1 **POST** `/patients/list`

List all patients for a doctor.

Request Body:

```
{
  "doctorId": "doc-001",
  "filter": "all",
  "search": ""
}
```

Field	Type	Required	Description
<code>doctorId</code>	string	yes	The doctor's ID returned from login
<code>filter</code>	string	no	One of: <code>all</code> , <code>critical</code> , <code>watch</code> , <code>low</code> . Default <code>all</code>
<code>search</code>	string	no	Search by patient name or ID

Success Response (200):

Returns only the fields needed for the dashboard patient cards. Full patient details are fetched via `POST /patients/get` when a card is clicked.

```
{
  "patients": [
    {
      "id": "pat-001",
      "name": "John Anderson",
      "age": 45,
      "sex": "M",
      "avatar": null,
      "symptom": "Mild chest discomfort, occasional fatig
e",
      "riskLevel": "low"
    },
    {
      "id": "pat-003",
      "name": "Robert Kim",
      "age": 71,
      "sex": "M",
      "avatar": null,
      "symptom": "Arrhythmia detected, shortness of breat
```

```

h",
    "riskLevel": "critical"
  }
]
}

```

Patient List Item Schema:

```

{
  "id": "string – unique patient identifier (e.g. pat-001)",
  "name": "string – full name",
  "age": "number – age in years",
  "sex": "string – 'M' or 'F'",
  "avatar": "string | null – URL to avatar image, or null",
  "symptom": "string – presenting symptoms summary",
  "riskLevel": "string – one of: 'low', 'watch', 'critical'"
}

```

Frontend location: `DashboardPage.tsx` — replaces `mockPatients` import. Also used by `PatientCard.tsx` for rendering.

3.2 **POST /patients/get**

Get a single patient's full record.

Request Body:

```

{
  "doctorId": "doc-001",
  "patientId": "pat-001"
}

```

Success Response (200):

```

{
  "id": "pat-003",
  "name": "Robert Kim",

```

```

"age": 71,
"sex": "M",
"avatar": null,
"symptom": "Arrhythmia detected, shortness of breath",
"bloodType": "B+",
"primaryCondition": "Atrial Fibrillation",
"vitals": {
  "heartRate": 98,
  "bloodPressure": "165/105",
  "oxygenSaturation": 92
},
"labResults": {
  "glucose": 142,
  "cholesterol": 245,
  "creatinine": 1.8
},
"riskLevel": "critical",
"riskPercentage": 85,
"riskHistory": [
  { "date": "2026-01-15", "score": 72 },
  { "date": "2026-01-22", "score": 78 },
  { "date": "2026-01-29", "score": 82 },
  { "date": "2026-02-05", "score": 85 }
],
"medications": [
  "Warfarin 5mg",
  "Metoprolol 50mg",
  "Digoxin 0.125mg",
  "Furosemide 40mg"
],
"allergies": ["Aspirin", "ACE Inhibitors"],
"clinicalSummary": "Persistent atrial fibrillation with rapid ventricular response despite rate control therapy. New-onset dyspnea on exertion concerning for decompensated heart failure. Recent echocardiogram shows reduced EF of 35%. INR therapeutic at 2.4. Urgent cardiology referral placed for possible cardioversion and advanced heart failure management. High stroke risk - CHA2DS2-VASc score of 5.",

```

```
"lastVisit": "2026-02-10"
}
```

(Same schema as the patient object in 3.1)

Error Response (404):

```
{
  "error": "Patient not found",
  "code": "PATIENT_NOT_FOUND"
}
```

Frontend locations:

- `PatientHubPage.tsx` — replaces `mockPatients.find((p) => p.id === id)`
- `ProcessingPage.tsx` — replaces `mockPatients.find((p) => p.id === mockReportData.patientId)`

3.3 **POST /patients/create**

Create a new patient record.

Request Body:

```
{
  "doctorId": "doc-001",
  "name": "Jane Doe",
  "age": 52,
  "sex": "F",
  "bloodType": "O+",
  "primaryCondition": "Hypertension",
  "symptom": "Headaches, elevated BP readings at home",
  "vitals": {
    "heartRate": 80,
    "bloodPressure": "148/95",
    "oxygenSaturation": 97
  },
  "labResults": {
    "glucose": 102,
    "cholesterol": 210,
    "creatinine": 1.0
  }
}
```

```
},
"medications": ["Lisinopril 10mg"],
"allergies": ["Codeine"],
"clinicalSummary": "New patient presenting with uncontrol
led hypertension..."
}
```

Success Response (201):

```
{
  "id": "pat-007",
  "name": "Jane Doe",
  "age": 52,
  "sex": "F",
  "avatar": null,
  "bloodType": "O+",
  "primaryCondition": "Hypertension",
  "symptom": "Headaches, elevated BP readings at home",
  "vitals": {
    "heartRate": 80,
    "bloodPressure": "148/95",
    "oxygenSaturation": 97
  },
  "labResults": {
    "glucose": 102,
    "cholesterol": 210,
    "creatinine": 1.0
  },
  "riskLevel": "watch",
  "riskPercentage": 0,
  "riskHistory": [],
  "medications": ["Lisinopril 10mg"],
  "allergies": ["Codeine"],
  "clinicalSummary": "New patient presenting with uncontrol
led hypertension...",
  "lastVisit": null
}
```


Note: `riskLevel`, `riskPercentage`, and `riskHistory` are computed server-side. On creation they can be set to defaults (`"low"`, `0`, `[]`) until the first diagnostic run.

Frontend location: `DashboardPage.tsx` — replaces the `alert('Add Patient')` FAB button.

4. Chat / Knowledge Graph

4.1 `POST /patients/chat/history`

Fetch existing chat/query history for a patient.

Request Body:

```
{
  "doctorId": "doc-001",
  "patientId": "pat-001"
}
```

Success Response (200):

```
{
  "messages": [
    {
      "id": "msg-1",
      "role": "user",
      "content": "What is the patient's cardiac history?",
      "timestamp": "2026-02-08T10:00:00Z"
    },
    {
      "id": "msg-2",
      "role": "assistant",
      "content": "Patient John Anderson has a documented history of stable angina pectoris, first diagnosed in 2024. No prior myocardial infarction. Stress test (Jan 2026) showed good exercise tolerance with no significant ST changes. Current management includes aspirin and statin therapy with good symptom control.",
      "timestamp": "2026-02-08T10:00:05Z"
    }
  ]
}
```

```
    }  
  ]  
}
```

Chat Message Schema:

```
{  
  "id": "string – unique message identifier",  
  "role": "string – 'user' or 'assistant'",  
  "content": "string – message text",  
  "timestamp": "string – ISO 8601 datetime"  
}
```

Frontend location: `PatientHubPage.tsx` — replaces `mockChatHistory[patient.id]` load in `useEffect`.

4.2 `POST /patients/chat/query`

Send a query about a patient to the Knowledge Graph / RAG pipeline.

Request Body:

```
{  
  "doctorId": "doc-001",  
  "patientId": "pat-001",  
  "message": "What is the patient's cardiac history?"  
}
```

Success Response (200):

```
{  
  "id": "msg-3",  
  "role": "assistant",  
  "content": "Patient John Anderson has a documented history of stable angina pectoris, first diagnosed in 2024...",  
  "timestamp": "2026-02-08T10:01:12Z"  
}
```

(Same chat message schema as 4.1, single message returned)

Frontend location: `PatientHubPage.tsx` — replaces the `setTimeout` fake AI response in `handleSendQuery`.

5. File Uploads

5.1 `POST /patients/uploads`

Upload clinical files (lab reports, clinical notes) for a patient.

Content-Type: `multipart/form-data`

Request Body (FormData):

Field	Type	Description
<code>doctorId</code>	string	The doctor's ID
<code>patientId</code>	string	The patient's ID
<code>files</code>	File[]	One or more files (PDF, JSON, TXT)

Success Response (200):

```
{
  "uploads": [
    {
      "uploadId": "upl-001",
      "fileName": "lab_report_feb2026.pdf",
      "fileSize": 245780,
      "mimeType": "application/pdf",
      "status": "uploaded"
    },
    {
      "uploadId": "upl-002",
      "fileName": "ecg_data.json",
      "fileSize": 12400,
      "mimeType": "application/json",
      "status": "uploaded"
    }
  ]
}
```

Upload Object Schema:

```
{
  "uploadId": "string – unique upload identifier",
  "fileName": "string – original file name",
  "fileSize": "number – file size in bytes",
  "mimeType": "string – MIME type of file",
  "status": "string – 'uploaded'"
}
```

Frontend location: `PatientHubPage.tsx` — replaces local-only `uploadedFiles` state. Files should be uploaded as they are dropped/selected.

6. Diagnostics Pipeline

6.1 `POST /diagnostics/run`

Kick off the agentic diagnostic pipeline (ML Swarm + MedGamma).

Request Body:

```
{
  "doctorId": "doc-001",
  "patientId": "pat-003",
  "observations": "Patient reports increased shortness of b
reath over the past 48 hours. Orthopnea noted. Lower extrem
ity edema observed during examination.",
  "uploadIds": ["upl-001", "upl-002"]
}
```

Success Response (202):

```
{
  "jobId": "diag-job-001",
  "patientId": "pat-003",
  "status": "processing",
  "startedAt": "2026-02-10T10:05:00Z"
}
```

Frontend location: `PatientHubPage.tsx` — replaces `handleRunDiagnostics` which currently just navigates to `/processing`. After this call, navigate to

`/processing/:jobId`.

6.2 **POST** `/diagnostics/status`

Poll processing status of a diagnostic job.

Request Body:

```
{
  "jobId": "diag-job-001"
}
```

Success Response (200):

```
{
  "jobId": "diag-job-001",
  "status": "processing",
  "progress": 65,
  "currentStep": "Running SHAP analysis",
  "startedAt": "2026-02-10T10:05:00Z"
}
```

Status enum values: `"queued"` | `"processing"` | `"complete"` | `"error"`

When `status` is `"complete"`:

```
{
  "jobId": "diag-job-001",
  "status": "complete",
  "progress": 100,
  "currentStep": "Done",
  "startedAt": "2026-02-10T10:05:00Z",
  "completedAt": "2026-02-10T10:06:42Z"
}
```

When `status` is `"error"`:

```
{
  "jobId": "diag-job-001",
  "status": "error",
}
```

```
"progress": 45,  
"currentStep": "Failed at feature extraction",  
"error": "Insufficient lab data to compute risk model",  
"startedAt": "2026-02-10T10:05:00Z"  
}
```

Frontend location: `ProcessingPage.tsx` — poll this while showing a loading state, then fetch the full report once `status === "complete"`.

7. Report Operations

7.1 `POST /diagnostics/report`

Fetch the full generated diagnostic report.

Request Body:

```
{  
  "jobId": "diag-job-001"  
}
```

Success Response (200):

```
{  
  "jobId": "diag-job-001",  
  "patientId": "pat-003",  
  "generatedAt": "2026-02-10T10:06:42Z",  
  "status": "draft",  
  "executiveSummary": "Based on comprehensive analysis of clinical data, laboratory results, and historical trends, the patient presents with elevated cardiovascular risk requiring immediate clinical attention. The integrated ML swarm analysis identified persistent atrial fibrillation with suboptimal rate control as the primary driver, compounded by progressive heart failure symptoms evidenced by reduced ejection fraction (35%) and new-onset dyspnea. Anticoagulation therapy appears adequate with therapeutic INR levels. However, the confluence of uncontrolled hypertension, elevated fasting glucose, and declining renal function creates a syner
```

gistic risk profile warranting aggressive intervention and close monitoring.",

"riskScore": 85,

"shapFeatures": [

{

"name": "Ejection Fraction (35%)",

"impact": -22,

"direction": "negative"

},

{

"name": "Atrial Fibrillation",

"impact": -18,

"direction": "negative"

},

{

"name": "Blood Pressure (165/105)",

"impact": -15,

"direction": "negative"

},

{

"name": "Fasting Glucose (142)",

"impact": -8,

"direction": "negative"

},

{

"name": "Age (71)",

"impact": -6,

"direction": "negative"

},

{

"name": "Therapeutic INR",

"impact": 5,

"direction": "positive"

},

{

"name": "No Prior MI",

"impact": 4,

"direction": "positive"

```

    }
  ],
  "featureHeatmap": [
    {
      "name": "Cardiac Function",
      "value": 0.92
    },
    {
      "name": "Blood Pressure",
      "value": 0.78
    },
    {
      "name": "Metabolic",
      "value": 0.65
    },
    {
      "name": "Renal Function",
      "value": 0.58
    },
    {
      "name": "Coagulation",
      "value": 0.25
    },
    {
      "name": "Lifestyle",
      "value": 0.45
    }
  ],
  "qualitativeFactors": {
    "riskFactors": [
      "Persistent AF",
      "Heart Failure",
      "Hypertension Stage 2",
      "Pre-diabetic"
    ],
    "protectiveFactors": [
      "Anticoagulated",
      "No smoking history",

```



```

        "Compliant with medications"
    ],
    "recommendations": [
        "Urgent cardiology referral",
        "Consider cardioversion",
        "Optimize rate control"
    ]
}

```

Full Report JSON Schema:

```

{
  "jobId": "string – diagnostic job identifier",
  "patientId": "string – patient identifier",
  "generatedAt": "string – ISO 8601 datetime",
  "status": "string – 'draft' or 'finalized'",

  "executiveSummary": "string – AI-generated narrative summary paragraph",

  "riskScore": "number – composite risk score 0-100",

  "shapFeatures": [
    {
      "name": "string – feature name with value in parentheses",
      "impact": "number – signed impact value (negative = increases risk, positive = decreases risk)",
      "direction": "string – 'positive', 'negative', or 'neutral'"
    }
  ],

  "featureHeatmap": [
    {
      "name": "string – clinical domain name",
      "value": "number – normalized risk value 0.0 to 1.0"
    }
  ]
}

```

```

(higher = more risk)"
    }
  ],

  "qualitativeFactors": {
    "riskFactors": ["string – identified risk factors"],
    "protectiveFactors": ["string – identified protective f
actors"],
    "recommendations": ["string – clinical recommendation
s"]
  }
}

```

Frontend location: `ProcessingPage.tsx` — replaces the entire `mockReportData` constant. Drives:

- Section A: Executive Summary text
- Section B: Risk score ring and percentage
- Section C: SHAP waterfall chart + Feature heatmap grid
- Section D: Qualitative factors JSON block

7.2 `POST /diagnostics/rebuild`

Re-generate the report incorporating physician notes/corrections.

Request Body:

```

{
  "jobId": "diag-job-001",
  "doctorId": "doc-001",
  "doctorNotes": "Patient started new medication yesterday,
which may affect BP readings. Also note family history of s
udden cardiac death."
}

```

Success Response (200):

Returns the **full updated report** (same schema as 7.1):

```

{
  "jobId": "diag-job-001",

```

```

    "patientId": "pat-003",
    "generatedAt": "2026-02-10T10:12:30Z",
    "status": "draft",
    "executiveSummary": "Updated executive summary incorporating physician's notes about new medication and family history...",
    "riskScore": 88,
    "shapFeatures": [
      { "name": "Ejection Fraction (35%)", "impact": -22, "direction": "negative" },
      { "name": "Family Hx Sudden Cardiac Death", "impact": -10, "direction": "negative" },
      { "name": "Atrial Fibrillation", "impact": -18, "direction": "negative" },
      { "name": "Blood Pressure (165/105)", "impact": -15, "direction": "negative" },
      { "name": "Fasting Glucose (142)", "impact": -8, "direction": "negative" },
      { "name": "Age (71)", "impact": -6, "direction": "negative" },
      { "name": "Therapeutic INR", "impact": 5, "direction": "positive" },
      { "name": "No Prior MI", "impact": 4, "direction": "positive" }
    ],
    "featureHeatmap": [
      { "name": "Cardiac Function", "value": 0.95 },
      { "name": "Blood Pressure", "value": 0.78 },
      { "name": "Metabolic", "value": 0.65 },
      { "name": "Renal Function", "value": 0.58 },
      { "name": "Coagulation", "value": 0.25 },
      { "name": "Lifestyle", "value": 0.45 }
    ],
    "qualitativeFactors": {
      "riskFactors": ["Persistent AF", "Heart Failure", "Hypertension Stage 2", "Pre-diabetic", "Family Hx Sudden Cardiac Death"],
      "protectiveFactors": ["Anticoagulated", "No smoking his

```

```
tory", "Compliant with medications"],  
  "recommendations": ["Urgent cardiology referral", "Consider cardioversion", "Optimize rate control", "Genetic screening for inherited cardiac conditions"]  
}  
}
```

Frontend location: `ProcessingPage.tsx` — replaces `handleRebuildReport` fake regeneration that just concatenates strings.

7.3 `POST /diagnostics/finalize`

Mark a report as physician-approved / finalized.

Request Body:

```
{  
  "jobId": "diag-job-001",  
  "doctorId": "doc-001"  
}
```

Success Response (200):

```
{  
  "jobId": "diag-job-001",  
  "status": "finalized",  
  "finalizedAt": "2026-02-10T10:20:00Z",  
  "finalizedBy": "doc-001"  
}
```

Frontend location: `ProcessingPage.tsx` — replaces `handleFinalize` which currently only flips local state.

7.4 `POST /diagnostics/export`

Download the report as a PDF file.

Request Body:

```
{  
  "jobId": "diag-job-001",
```

```
"format": "pdf",  
"type": "draft"  
}
```

Field	Type	Description
<code>jobId</code>	string	Diagnostic job ID
<code>format</code>	string	<code>pdf</code> (only supported format for now)
<code>type</code>	string	<code>draft</code> or <code>final</code> — determines PDF styling

Success Response (200):

```
Content-Type: application/pdf  
Content-Disposition: attachment; filename="report-pat-003-d  
iag-job-001.pdf"  
  
<binary PDF data>
```

Error Response (400):

```
{  
  "error": "Report must be finalized before exporting as fi  
nal",  
  "code": "REPORT_NOT_FINALIZED"  
}
```

Frontend location: `ProcessingPage.tsx` — replaces both:

- "Download Preliminary Draft" button → `POST /diagnostics/export` with `type: "draft"`
- "Export Final PDF" button → `POST /diagnostics/export` with `type: "final"`

7.5 `POST /diagnostics/commit`

Save the finalized report to the patient's permanent record (PostgreSQL Vector DB).

Request Body:

```
{  
  "jobId": "diag-job-001",  
}
```

```
"doctorId": "doc-001"
}
```

Success Response (200):

```
{
  "success": true,
  "recordId": "rec-2026-0210-001",
  "patientId": "pat-003",
  "committedAt": "2026-02-10T10:25:00Z",
  "committedBy": "doc-001"
}
```

Error Response (400):

```
{
  "error": "Report must be finalized before committing to record",
  "code": "REPORT_NOT_FINALIZED"
}
```

Frontend location: `ProcessingPage.tsx` — replaces `handleSaveToRecord` which currently just shows an `alert()`.

Appendix: Summary of All Hardcoded Data to Replace

Hardcoded Source	File	Replaced By Endpoint
<code>mockDoctor</code>	<code>data/patients.ts</code>	<code>POST /login</code> (returned in response)
<code>mockPatients</code> (list)	<code>data/patients.ts</code>	<code>POST /patients/list</code>
<code>mockPatients.find()</code>	<code>PatientHubPage.tsx</code>	<code>POST /patients/get</code>
<code>mockPatients.find()</code>	<code>ProcessingPage.tsx</code>	<code>POST /patients/get</code>
<code>mockNotifications</code>	<code>data/patients.ts</code>	<code>POST /notifications</code>
<code>mockChatHistory</code>	<code>data/patients.ts</code>	<code>POST /patients/chat/history</code>
Fake AI chat response	<code>PatientHubPage.tsx</code>	<code>POST /patients/chat/query</code>
Local-only file state	<code>PatientHubPage.tsx</code>	<code>POST /patients/uploads</code>

Hardcoded Source	File	Replaced By Endpoint
Direct navigate()	PatientHubPage.tsx	POST /diagnostics/run
mockReportData	ProcessingPage.tsx	POST /diagnostics/report
Fake string concatenation	ProcessingPage.tsx	POST /diagnostics/rebuild
Local-only finalize state	ProcessingPage.tsx	POST /diagnostics/finalize
alert('Exporting...')	ProcessingPage.tsx	POST /diagnostics/export
alert('Saving...')	ProcessingPage.tsx	POST /diagnostics/commit
alert('Add Patient')	DashboardPage.tsx	POST /patients/create
setTimeout fake login	LoginPage.tsx	POST /login

Appendix: Error Response Format

All error responses follow this consistent shape:

```
{
  "error": "string – human-readable error message",
  "code": "string – machine-readable error code"
}
```

Standard HTTP status codes used:

Code	Meaning
200	Success
201	Created (new resource)
202	Accepted (async job)
400	Bad request
401	Invalid credentials
404	Not found
500	Internal server error

Appendix: Route Changes Needed

The frontend router (`router.tsx`) needs updating:

Current Route	Updated Route	Reason
<code>/processing</code>	<code>/processing/:jobId</code>	Needs job ID to fetch report

The `PatientHubPage` `handleRunDiagnostics` should navigate to `/processing/:jobId` after receiving the job ID from `POST /diagnostics/run`.